

Desenvolvedor(a) Back End Node

T1 4: Mini-Projeto Avaliativo - Módulo 01 - Semana 09

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	3
4. REQUISITOS DAS TAREFAS	3
4.1. ORGANIZAÇÃO ARQUITETURAL DO PROJETO	3
4.2. Requisitos funcionais	5
RF01 – Configurar o projeto com Node.js e TypeScript	5
RF02 – Criar uma interface para o Pokémon resumido	6
RF03 – Criar uma interface para o retorno da API	6
RF04 – Buscar Pokémon na PokeAPI	7
RF05 – Tratar erro de Pokémon inexistente	7
RF06 – Mapear o retorno da API para um objeto simplificado	8
RF07 – Criar um catálogo local em memória	8
RF08 – Adicionar Pokémon ao catálogo	9
RF09 – Listar Pokémon do catálogo	9
RF10 – Remover Pokémon do catálogo pelo ID	9
RF11 – Usar métodos de array	10
RF12 – Criar uma classe simples	11
RF13 – Demonstrar o fluxo no main.ts	12
4.3. Requisitos técnicos	13
4.4. Organização com Kanban e GitHub	15
4.5. Versionamento com GitHub	16
4.6. Registro de testes no README	17
4.7. GRAVAÇÃO DE VÍDEO	18
5. CRITÉRIOS DE AVALIAÇÃO	18
Apresentação do Projeto – 4,00 pontos	19
Desenvolvimento do Projeto – 6,00 pontos	20
6. Checklist final de entrega	20
7. Modelo de README.md para os alunos	21

1. CONTEXTUALIZAÇÃO

O desenvolvimento back-end exige que a pessoa programadora saiba criar aplicações capazes de executar regras de negócio, manipular dados, consumir informações externas e tratar erros sem interromper o funcionamento do sistema.

No contexto de aplicações modernas, é comum que sistemas back-end consultem APIs externas, recebam dados em formato JSON, transformem essas informações em estruturas internas mais simples e disponibilizem resultados organizados para outras partes da aplicação.

Neste mini-projeto, você irá desenvolver uma aplicação simples em Node.js com TypeScript, executada pelo terminal, para consultar dados de Pokémon em uma API pública e organizar alguns resultados em um catálogo local durante a execução do programa.

O foco do projeto é praticar os conteúdos estudados até esta etapa do curso:

- JavaScript no back-end com Node.js;
- lógica de programação;
- funções;
- arrays e objetos;
- JSON;
- métodos de array;
- callbacks, Promises e async/await;
- Programação Orientada a Objetos;
- migração de JavaScript para TypeScript;
- tipagem básica;
- interfaces;
- classes;
- métodos;
- construtores;
- modificadores de acesso;
- fetch;
- tipagem de respostas de API;
- GitHub;
- GitFlow;

A documentação oficial do TypeScript apresenta tipos primitivos como *string*, *number* e *boolean*, *arrays* tipados, funções com tipagem de parâmetros e retorno, *object types* e propriedades opcionais. Esses elementos serão utilizados neste mini-projeto de forma introdutória e prática.

A documentação oficial do Node.js apresenta `fetch` como uma implementação compatível com a API de navegador e permite o uso de `await fetch(...)` para buscar dados externos. A PokeAPI será utilizada porque possui um endpoint público para consulta de Pokémon por nome ou ID.

2. DESAFIO

Você foi contratado como pessoa desenvolvedor back-end **júnior** para criar uma ferramenta simples de consulta e organização de Pokémon. O sistema deverá se chamar:

Pokédex TypeScript Lite

A aplicação deverá consultar a PokeAPI, buscar um Pokémon pelo nome ou ID, transformar os dados retornados em um objeto simplificado e permitir que alguns Pokémon sejam adicionados a um catálogo local.

O projeto será executado pelo terminal usando Node.js e TypeScript.

Objetivo técnico do projeto

Criar uma aplicação back-end simples, sem interface gráfica, capaz de:

- consultar uma API externa;
- tratar resposta JSON;
- mapear dados externos para um objeto TypeScript;
- armazenar os Pokémon em um arquivo local;
- listar os Pokémon salvos;
- impedir registros duplicados;
- remover Pokémon do catálogo;
- exibir mensagens claras no terminal;
- documentar instalação, execução e exemplos de uso no README.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do mini-projeto, o estudante ou squad de até 3 integrantes deverá entregar:

- **Repositório público no GitHub:** link público contendo todo o histórico de desenvolvimento da aplicação em sua conta pessoal do GitHub;

- **Projeto em Node.js com TypeScript (Código Fonte Tipado e Modular):** Arquivos .ts organizados rigorosamente conforme o padrão de camadas estipulado, sem arquivos lixo;
 - **Base de Dados Inicial:** Arquivo `pc_box.json` presente no repositório, inicializado com `[]` (array vazio), gerado de forma limpa pelo fluxo da aplicação;
- **Arquivo README.md completo** (Documentação Técnica): arquivo README.md detalhado descrevendo os comandos de instalação das dependências de desenvolvimento, arquitetura adotada e scripts;
- **Histórico de commits semânticos** no GitHub: registro de evolução lógica utilizando boas práticas de mensagens no Git.
- código executável pelo terminal;
- exemplos de execução documentados no README.

Entrega no AVA

A entrega deverá ser **submetida na tarefa do AVA:** Módulo 01 - Mini-Projeto Avaliativo

- **Prazo de entrega:** 09/10/26 até às 22h
- **Peso:** 25% da nota do Módulo 01, caso esse seja o peso oficial da matriz.

Links obrigatórios

O estudante deverá **enviar no AVA:**

- **link do repositório** público no GitHub;
- **link do vídeo** demonstrando exemplos de execução do sistema

4. REQUISITOS DAS TAREFAS

4.1. ORGANIZAÇÃO ARQUITETURAL DO PROJETO

A aplicação deve rejeitar códigos monolíticos. Todas as responsabilidades precisam estar rigidamente isoladas em camadas, adotando a tipagem forte e a injeção de dependências. A árvore estrutural exigida é:

```
/pasta_raiz_do_projeto
|
```

```

├─ src/
│   └─ main.ts # Ponto de entrada. Instancia os serviços, injeta as dependências e
│       inicia o loop principal do menu.
│
│   └─ controllers/
│       └─ TerminalController.ts # Camada de Interface do Usuário. Gerencia entrada
do terminal e orquestra exibições.
│
│   └─ services/
│       └─ PokeApiService.ts # Camada de Integração Externa (fetch nativo). Retorna
Promises tipadas com Interfaces.
│       └─ BoxService.ts # Camada de Persistência Local (node:fs/promises). Aplica
métodos funcionais validados pelo TS.
│
│   └─ models/
│       └─ Pokemon.ts # Interfaces/Types e Classes de Entidade. Molde rigoroso dos
atributos consumidos da API.
│       └─ CustomErrors.ts # Classes de Exceções Customizadas que estendem Error (ex:
APIError, LocalBoxError).
│
│   └─ utils/
│       └─ textFormatters.ts # Funções utilitárias puras com tipagem explícita de
parâmetros e retorno.
│
├─ pc_box.json # Banco de dados local baseado em persistência de arquivos JSON.
├─ tsconfig.json # Manual de regras rígidas do compilador TypeScript (Strict Mode).
└─ package.json # Manifesto de configuração (contendo scripts de compilação/execução
e devDependencies).

```

A nomenclatura e organização não necessita ser **EXATAMENTE** como o exemplo acima. O(a) aluno(a) pode separar em mais arquivos ou pastas conforme achar necessário.

O escopo do projeto consiste em projetar uma aplicação utilitária em Node.js com TypeScript puro que sirva como ponte de dados assíncrona entre a rede externa e o sistema de arquivos local do servidor, mitigando falhas de tipagem e impedindo estados inconsistentes na base local (**pc_box.json**).

A fonte de dados externa do sistema é a **PokeAPI**, uma API pública, gratuita e sem necessidade de autenticação. A URL base para busca de espécies por nome ou ID é:

- <https://pokeapi.co/api/v2/pokemon/{nome-ou-id}>

Exemplos de requisições válidas:

- <https://pokeapi.co/api/v2/pokemon/pikachu>
- <https://pokeapi.co/api/v2/pokemon/25>

O retorno é um objeto JSON. Para este projeto, os campos relevantes que devem ser mapeados via Interface são:

id — identificador numérico oficial

name — nome da espécie

types — array de objetos contendo o campo **type.name**

stats — array de objetos contendo **stat.name** e **base_stat** (extrair HP, Attack e Defense)

Requisições para nomes inexistentes retornam status 404 e devem ser tratadas pelo bloco try/catch do `PokeApiService`.

OBS: Documentação oficial da PokeAPI: <https://pokeapi.co/docs/v2> Consulte a seção *Pokémon* para visualizar a estrutura completa do objeto de resposta e identificar os campos que serão mapeados nas suas Interfaces.

Maneira de Execução

A aplicação deve ser iniciada e mantida estritamente via terminal de comandos, utilizando o runtime do Node.js através do ponto de entrada centralizado:

npm run start

4.2. Requisitos funcionais

RF01 – Configurar o projeto com Node.js e TypeScript

O estudante deverá criar um projeto Node.js com TypeScript. O projeto deverá conter:

package.json
tsconfig.json
src/main.ts
README.md

Scripts sugeridos no `package.json`:

```
{
  "scripts": {
    "dev": "tsx src/main.ts",
    "build": "tsc"
  }
}
```

Dependências de desenvolvimento sugeridas:

```
typescript
tsx
@types/node
```

O uso de **tsx** facilita a execução em ambiente de desenvolvimento, mas o estudante também poderá executar conforme orientação do docente.

RF02 – Criar uma interface para o Pokémon resumido

O estudante deverá criar pelo menos uma **interface** ou **type** para representar o Pokémon simplificado utilizado no projeto. Exemplo:

```
export interface PokemonResumo {
  id: number;
  nome: string;
  tipos: string[];
  altura: number;
  peso: number;
}
```

O TypeScript permite especificar tipos para objetos, funções e arrays, o que ajuda a descrever melhor os dados manipulados no código. ([TypeScript](#))

RF03 – Criar uma interface para o retorno da API

O estudante deverá criar uma interface simples para representar apenas os campos da PokeAPI que serão usados. Exemplo:

```
interface PokemonApiResponse {  
  id: number;  
  name: string;  
  height: number;  
  weight: number;  
  types: {  
    type: {  
      name: string;  
    };  
  }[];  
}
```

Não é necessário mapear todos os campos da API. A PokeAPI retorna um objeto grande, mas o projeto deve usar apenas os campos necessários para o mini-projeto.

RF04 – Buscar Pokémon na PokeAPI

Criar uma função assíncrona para buscar Pokémon por nome ou ID. Exemplo de assinatura:

```
async function buscarPokemon(nomeOuId: string): Promise<PokemonResumo | null> {  
  // implementação  
}
```

A função deverá:

- receber uma **string**;
- usar **fetch**;
- consultar a PokeAPI;
- tratar erro quando o Pokémon não existir;
- retornar **null** em caso de erro;
- retornar um objeto **PokemonResumo** em caso de sucesso.

O TypeScript permite anotar o retorno de funções assíncronas com **Promise<T>**, o que se conecta diretamente ao uso de **async/await** estudado na turma. ([TypeScript](#))

RF05 – Tratar erro de Pokémon inexistente

Quando o Pokémon não existir, o sistema não deve quebrar.

Exemplo de comportamento esperado:

```
[ERRO] Pokémon não encontrado: pokemon-inexistente
```

O estudante deverá usar `try/catch` na função de busca. Exemplo de lógica:

```
try {
  const resposta = await fetch(url);
  if (!resposta.ok) {
    console.log("[ERRO] Pokémon não encontrado.");
    return null;
  }

  const dados = await resposta.json();
  // mapear dados

} catch (erro) {
  console.log("[ERRO] Não foi possível buscar o Pokémon.");
  return null;
}
```

Não é necessário criar classes de erro customizadas.

RF06 – Mapear o retorno da API para um objeto simplificado

A função de busca deverá transformar a resposta da API em um objeto simples.

Campos obrigatórios:

```
id
nome
tipos
altura
peso
```

Exemplo de objeto final:

```
{
  id: 25,
  nome: "pikachu",
  tipos: ["electric"],
  altura: 4,
  peso: 60
}
```

Exemplo de mapeamento dos tipos:

```
const tipos = dados.types.map((item) => item.type.name);
```

RF07 – Criar um catálogo local em memória

O estudante deverá criar um array local para armazenar os Pokémon buscados.

Exemplo:

```
let catalogo: PokemonResumo[] = [];
```

Esse catálogo será mantido apenas durante a execução do programa. Não será obrigatório salvar dados em arquivo.

RF08 – Adicionar Pokémon ao catálogo

Criar uma função para adicionar Pokémon ao catálogo. Exemplo de assinatura:

```
function adicionarAoCatalogo(catalogo:
PokemonResumo[], pokemon: PokemonResumo
): PokemonResumo[] {
  // implementação
}
```

Regra obrigatória: não permitir Pokémon duplicado pelo mesmo **id**.

Exemplo de comportamento:

```
[OK] pikachu adicionado ao catálogo.
```

Caso já exista:

[AVISO] pikachu já está no catálogo.

RF09 – Listar Pokémon do catálogo

Criar uma função para listar os Pokémon salvos no catálogo. Exemplo de assinatura:

```
function listarCatalogo(catalogo: PokemonResumo[]): void {  
  // implementação  
}
```

Exemplo de saída:

```
Catálogo atual:  
#25 - pikachu | Tipos: electric | Altura: 4 | Peso: 60  
#4 - charmander | Tipos: fire | Altura: 6 | Peso: 85
```

Caso o catálogo esteja vazio:

[AVISO] Catálogo vazio.

RF10 – Remover Pokémon do catálogo pelo ID

Criar uma função para remover um Pokémon pelo ID. Exemplo de assinatura:

```
function removerDoCatalogo(  
  catalogo: PokemonResumo[],  
  id: number  
): PokemonResumo[] {  
  // implementação  
}
```

A função deverá:

- receber o catálogo;
- receber o ID;
- verificar se o Pokémon existe;

- remover o Pokémon, se existir;
- retornar o catálogo atualizado;
- exibir mensagem clara no terminal.

Exemplo de saída:

```
[OK] Pokémon removido do catálogo.
```

Caso não exista:

```
[AVISO] Nenhum Pokémon encontrado com esse ID.
```

RF11 – Usar métodos de array

O projeto deverá usar pelo menos **3 métodos de array** entre:

```
map  
filter  
find  
some  
every  
reduce  
forEach
```

Sugestões:

Método	Uso possível
map	transformar types da API em lista de nomes
find	verificar se um Pokémon já existe no catálogo
some	verificar duplicidade
filter	remover Pokémon pelo ID
forEach	listar catálogo
every	validar se todos os Pokémon têm nome
reduce	calcular peso total ou média de peso

RF12 – Criar uma classe simples

O estudante deverá criar uma classe simples para organizar alguma parte do sistema. Sugestão:

```
class CatalogoPokemon {
  private pokemons: PokemonResumo[] = [];

  adicionar(pokemon: PokemonResumo): void {
    const jaExiste = this.pokemons.some((item) => item.id ===
pokemon.id);

    if (jaExiste) {
      console.log(`[AVISO] ${pokemon.nome} já está no catálogo.`);
      return;
    }

    this.pokemons.push(pokemon);
    console.log(`[OK] ${pokemon.nome} adicionado ao catálogo.`);
  }

  listar(): void {
    if (this.pokemons.length === 0) {
      console.log("[AVISO] Catálogo vazio.");
      return;
    }

    this.pokemons.forEach((pokemon) => {
      console.log(
        `#${pokemon.id} - ${pokemon.nome} | Tipos:
        ${pokemon.tipos.join(", ")}`
      );
    });
  }

  remover(id: number): void {
```

```
    const existe = this.pokemons.some((pokemon) => pokemon.id === id);

    if (!existe) {
        console.log("[AVISO] Nenhum Pokémon encontrado com esse ID.");
        return;
    }

    this.pokemons = this.pokemons.filter((pokemon) => pokemon.id !== id);
    console.log("[OK] Pokémon removido do catálogo.");
  }
}
```

Esse requisito avalia:

- classe;
- atributo;
- método;
- construtor, caso utilizado;
- modificador de acesso;
- uso de array dentro de classe.

Não será obrigatório usar herança nesta entrega mínima.

RF13 – Demonstrar o fluxo no `main.ts`

O arquivo `main.ts` deverá demonstrar o funcionamento do projeto. Exemplo:

```
async function main() {

    const catalogo = new CatalogoPokemon();

    const pikachu = await buscarPokemon("pikachu");

    if (pikachu !== null) {
        catalogo.adicionar(pikachu);
    }

    const charmander = await buscarPokemon("charmander");

    if (charmander !== null) {
        catalogo.adicionar(charmander);
    }

    const pikachuDuplicado = await buscarPokemon("pikachu");

    if (pikachuDuplicado !== null) {
        catalogo.adicionar(pikachuDuplicado);
    }

    await buscarPokemon("pokemon-inexistente");
    catalogo.listar();
    catalogo.remover(25);
    catalogo.listar();

}

main();
```

Não será obrigatório criar menu interativo.

4.3. Requisitos técnicos

Conteúdo	Obrigatório?	Como demonstrar
Node.js	Sim	Executar projeto no back-end
JavaScript no back-end	Sim	Uso do Node como ambiente
TypeScript	Sim	Arquivos <code>.ts</code>
<code>tsconfig.json</code>	Sim	Configuração básica do projeto
Tipos primitivos	Sim	<code>string</code> , <code>number</code> , <code>boolean</code> se necessário
Arrays	Sim	Catálogo de Pokémon
Objetos	Sim	Pokémon resumido
JSON	Sim	Resposta da API
Funções	Sim	Buscar, adicionar, listar e remover
Arrow functions	Sim	Métodos de array
Métodos de array	Sim	Pelo menos 3 métodos
Interfaces ou types	Sim	<code>PokemonResumo</code> e resposta da API
Parâmetros tipados	Sim	Funções com tipos
Retornos tipados	Sim	<code>`Promise<PokemonResumo>`</code>
Classes	Sim	Classe <code>CatalogoPokemon</code>
Atributos	Sim	Array interno do catálogo
Métodos	Sim	<code>adicionar</code> , <code>listar</code> , <code>remover</code>
Modificadores de acesso	Sim	<code>private</code> ou <code>public</code>
Herança	Não obrigatório	Pode ser desafio extra
Generics	Não obrigatório	Pode ser desafio extra
Union types	Não obrigatório	Pode aparecer em <code>`PokemonResumo`</code>
Intersection types	Não	Não cobrar

Conteúdo	Obrigatório?	Como demonstrar
<code>fetch</code>	Sim	Consulta à PokeAPI
Promises	Sim	Retorno da função assíncrona
<code>async/await</code>	Sim	Busca na API
<code>try/catch</code>	Sim	Tratamento de erro
<code>fs/promises</code>	Não	Não cobrar
<code>readline</code>	Não	Não cobrar
Express/Nest/Fastify	Não	Não cobrar
Banco de dados	Não	Não cobrar
GitHub	Sim	Repositório público
GitFlow	Sim	Branches simples
Kanban	Sim	Planejamento visível
Vídeo	Não	Substituído por exemplos no README

4.4. Organização com Kanban e GitHub

O estudante ou squad deverá organizar o trabalho em um quadro Kanban. Colunas mínimas:

Backlog A Fazer Em Andamento Concluído

Ferramentas aceitas:

- Trello;
- GitHub Projects;
- Notion;
- planilha;
- arquivo no README;

- imagem do quadro.

Tarefas sugeridas para o Kanban:

```
Criar repositório no GitHub
Configurar projeto Node com TypeScript
Criar package.json
Criar tsconfig.json
Criar src/main.ts
Criar interface PokemonResumo
Criar interface PokemonApiResponse
Criar função buscarPokemon
Usar fetch para consultar a PokeAPI
Tratar erro de Pokémon inexistente
Mapear resposta da API
Criar classe CatalogoPokemon
Criar método adicionar
Bloquear Pokémon duplicado
Criar método listar
Criar método remover
Usar pelo menos 3 métodos de array
Testar fluxo no main.ts
Atualizar README.md
Registrar exemplos de execução no README
Enviar links no AVA
```

4.5. Versionamento com GitHub

Branches mínimas

Para projeto individual:

```
main
develop
feat/pokedex
docs/readme
```

Para squad (pequeno grupo de até 03 pessoas):

```
main
develop
feat/api-pokemon
feat/catalogo
docs/readme
```

Commits mínimos

- Projeto individual: mínimo de 04 commits
- Projeto em squad (pequeno grupo de até 03 pessoas): mínimo de 06 commits

Exemplos de commits

```
feat: configura projeto com typescript
feat: cria interfaces de pokemon
feat: implementa busca na pokeapi
feat: cria classe de catalogo
feat: adiciona validacao de duplicidade
fix: trata pokemon inexistente
docs: atualiza readme com instrucoes
```

4.6. Registro de testes no README

Como a entrega do vídeo será removida, o estudante deverá registrar no README exemplos de execução. O README deverá conter:

1. descrição do projeto;
2. objetivo;
3. tecnologias utilizadas;
4. pré-requisitos;
5. como instalar;
6. como executar;
7. funcionalidades;
8. exemplos de execução;
9. explicação curta dos arquivos;
10. link do Kanban;
11. branches utilizadas.

Exemplos mínimos obrigatórios no README

Busca válida

Entrada testada:

pikachu

Saída esperada:

[OK] Pokémon encontrado: pikachu

#25 - pikachu | Tipos: electric | Altura: 4 | Peso: 60

Busca inválida

Entrada testada:

pokemon-inexistente

Saída esperada:

[ERRO] Pokémon não encontrado.

Duplicidade

Entrada testada:

adicionar pikachu duas vezes

Saída esperada:

[AVISO] pikachu já está no catálogo.

Remoção

Entrada testada:

remover ID 25

Saída esperada:

[OK] Pokémon removido do catálogo.

4.7. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento das tarefas você deverá gravar um vídeo (ou seu grupo de até 03 alunos, um vídeo por grupo), com tempo **máximo de 5 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?

- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação e que seja possível ouvir sua voz bem. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link (ou subo via YouTube como vídeo não listado), e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no `readme.md` do seu projeto no repositório do GitHub. É um requisito que qualquer pessoa consiga visualizar seu vídeo via algum link.

5. CRITÉRIOS DE AVALIAÇÃO

A nota varia de **0 a 10 pontos**. O mini-projeto possui peso de **25% sobre a nota do Módulo 01**, caso esse seja o peso oficial da matriz.

Serão desconsiderados e receberão nota **0** os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. O estudante pode consultar documentação, exemplos e ferramentas de apoio, desde que compreenda, adapte e consiga explicar o código entregue.

Uso do GitHub e Readme.md				
Nº	Critério de Avaliação	0	0,75	1,50
01	Versionamento com branches e commits	O repositório do projeto não apresenta branches e commits.	O repositório do projeto apresenta parte das branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.	O repositório do projeto apresenta branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.
Nº	Critério de Avaliação	0	0,25	0,50
02	Organização dos arquivos no repositório	O repositório do projeto não apresenta os arquivos estruturados conforme as instruções.	O repositório do projeto apresenta parte dos arquivos estruturados conforme as instruções.	O repositório do projeto apresenta os arquivos estruturados conforme as instruções com Readme.md estruturado

Apresentação do Projeto

Nº	Critério de Avaliação	0,00	1,00	2,00
03	Gravação de vídeo	Não gravou o vídeo de apresentação.	Gravou o vídeo de apresentação, mas não apresentou todos os requisitos do item 4.7	O vídeo apresentou todos os requisitos do item 4.6.

Desenvolvimento do Projeto

Nº	Critério de Avaliação	0,00	0,50	1,00
04	Configuração do Ambiente e Execução	O projeto não inclui a configuração do TypeScript (falta <code>tsconfig.json</code> ou <code>package.json</code>).	O projeto executou, porém faltam configurações básicas no <code>tsconfig.json</code> ou scripts essenciais no <code>package.json</code> (ex.: script para compilar/rodar).	Configurou projeto Node.js com TypeScript e executa sem erro: Node.js, TypeScript, package.json, tsconfig, scripts e execução via terminal
05	Tipagem e Contratos de Dados	Não criou <code>interfaces</code> ou <code>types</code> para representar as entidades do projeto.	Criou algumas <code>interfaces/types</code> , mas deixou de tipar parâmetros de funções, retornos ou propriedades importantes.	Criou interfaces/types e aplicou tipagem em funções, parâmetros e retornos
06	Consumo e Tratamento da PokeAPI	Não realizou a integração com a PokeAPI ou utilizou chamadas síncronas/bloqueantes.	Realizou a busca com <code>fetch</code> e <code>async/await</code> , mas não tratou o erro 404 (quando o Pokémon não existe) ou falta o bloco <code>try/catch</code> .	Implementou Integração com PokeAPI: busca na PokeAPI com <code>fetch</code> , <code>async/await</code> e <code>try/catch</code> (tratamento de JSON e erro 404)
07	Transformação e Mapeamento de Dados	Não realizou a conversão dos dados, retornando a resposta bruta e complexa da PokeAPI.	Realizou o mapeamento, mas omitiu campos essenciais ou retornou dados com estruturas incorretas/incompletas.	Mapeou corretamente o retorno da API para um objeto simplificado
08	Arquitetura e Modularização do Código	Todo o código está concentrado em um único arquivo (código monolítico/script único) e não houve divisão de pastas ou	Criou a estrutura de pastas/camadas (<code>controllers</code> , <code>services</code> , etc.), mas	Estruturou a arquitetura da aplicação de forma modular, dividindo as responsabilidades do código em camadas lógicas (<code>controllers</code> ,

		separação clara de responsabilidades.	misturou as responsabilidades.	services, models e utils) para evitar código monolítico.
09	Gerenciamento do Catálogo Local	Não implementou as funções ou métodos para cadastrar, listar ou remover Pokémons do catálogo local ou o catálogo local não armazenou dados em memória de forma estruturada ou o código apresentou erros de execução ao tentar realizar operações básicas no catálogo.	As funcionalidades funcionam, porém não há validação para evitar registros duplicados , permitindo a inserção de Pokémons repetidos no catálogo ou há inconsistências na manipulação dos dados em memória ou falhas ao identificar Pokémons por ID/home.	Implementou as funcionalidades do catálogo local de Pokémons para cadastrar, listar e remover itens, aplicando validações para evitar a inclusão de registros duplicados.

6. Checklist final de entrega

Antes de enviar no AVA, confira:

- [] Criei o repositório público no GitHub
- [] Configurei o projeto Node.js com TypeScript
- [] Criei o package.json
- [] Criei o tsconfig.json
- [] Criei o arquivo src/main.ts
- [] Criei uma interface ou type para PokemonResumo
- [] Criei uma interface simples para a resposta da PokeAPI
- [] Criei função assíncrona para buscar Pokémon
- [] Usei fetch
- [] Usei async/await
- [] Usei try/catch
- [] Tratei erro de Pokémon inexistente
- [] Transformei o retorno da API em objeto simples
- [] Criei catálogo local em array ou classe
- [] Criei função ou método para adicionar Pokémon
- [] Impedi Pokémon duplicado
- [] Criei função ou método para listar catálogo
- [] Criei função ou método para remover Pokémon por ID

```
[ ] Usei pelo menos 3 métodos de array
[ ] Exibi mensagens claras no terminal
[ ] Testei o fluxo no main.ts
[ ] Atualizei o README.md
[ ] Incluí exemplos de execução no README
[ ] Fiz commits no GitHub
[ ] Usei branches mínimas
[ ] Enviei o link do GitHub no AVA
[ ] Enviei o link do vídeo no AVA
[ ] Não deixei vídeo como item obrigatório
```

7. Modelo de README.md para os alunos

```
# Pokédex TypeScript Lite
```

```
## Sobre o projeto
```

O Pokédex TypeScript Lite é uma aplicação simples em Node.js com TypeScript que consulta dados de Pokémon na PokeAPI e organiza alguns resultados em um catálogo local durante a execução do programa.

```
## Objetivo
```

Praticar os principais conceitos do Módulo 01:

- Node.js;
- JavaScript no back-end;
- TypeScript;
- interfaces;
- funções tipadas;
- arrays;
- objetos;
- JSON;
- métodos de array;
- classes;
- async/await;
- fetch;
- tratamento de erros;
- GitHub;

- GitFlow;
- Kanban.

Tecnologias utilizadas

- Node.js
- TypeScript
- TSX
- PokeAPI
- Git
- GitHub

Pré-requisitos

Antes de executar o projeto, é necessário ter instalado:

- Node.js
- npm
- Git

Como instalar

Clone o repositório:

```
```bash
git clone LINK_DO_REPOSITORIO
```

Acesse a pasta do projeto:

```
cd pokedex-typescript-lite
```

Instale as dependências:

```
npm install
```

Como executar

Execute o projeto em ambiente de desenvolvimento:

```
npm run dev
```

Estrutura do projeto

```
pokedex-typescript-lite/
```

```
|
```

```
└─ src/
```

```
| |— main.ts
| |— types.ts
| |— pokeApi.ts
| |— catalogo.ts
|
```

```
|— package.json
|— tsconfig.json
|— README.md
```

## Funcionalidades

- Buscar Pokémon por nome ou ID
- Tratar erro de Pokémon inexistente
- Transformar resposta da API em objeto simplificado
- Adicionar Pokémon ao catálogo local
- Impedir Pokémon duplicado
- Listar catálogo
- Remover Pokémon por ID
- Exibir mensagens no terminal
- Exemplos de execução
- Busca válida

Entrada testada:

pikachu

Saída obtida:

[OK] Pokémon encontrado: pikachu

#25 - pikachu | Tipos: electric | Altura: 4 | Peso: 60

Busca inválida

Entrada testada:

pokemon-inexistente

Saída obtida:

[ERRO] Pokémon não encontrado.

Duplicidade

Entrada testada:

adicionar pikachu duas vezes

Saída obtida:

[AVISO] pikachu já está no catálogo.

Remoção

Entrada testada:

remover ID 25

Saída obtida:

[OK] Pokémon removido do catálogo.

Conceitos aplicados

TypeScript

Explique onde foram utilizados tipos, interfaces, parâmetros e retornos tipados.

Interface PokemonResumo

Explique o objetivo da interface criada para representar os dados simplificados do Pokémon.

Fetch e async/await

Explique como a aplicação consulta a PokeAPI.

Tratamento de erros

Explique como o projeto lida com Pokémon inexistente ou erro de busca.

Métodos de array

Informe onde foram usados map, filter, find, some, every, reduce ou forEach.

Classe CatalogoPokemon

Explique quais atributos e métodos foram criados.

Organização do Kanban

Link do Kanban:

COLE\_AQUI\_O\_LINK

Branches utilizadas

- main
- develop
- feat/pokedex
- docs/readme

Melhorias futuras

- Criar menu interativo no terminal
- Salvar catálogo em arquivo JSON
- Exibir HP, ataque e defesa
- Criar filtros por tipo de Pokémon
- Criar uma API própria com Express